

Contextualizing agile software development

Philippe Kruchten^{*,†}

Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, BC, Canada

SUMMARY

This paper presents a contextual model for software-intensive systems development to guide the adoption and adaptation of agile software development practices. This model was found especially useful when the project context departs significantly from the “agile sweet spot”, that is, the ideal conditions in which agile software development practices originated from, and where they are most likely to succeed, “out of the box”. This is the case for large systems, distributed development environment, safety-critical systems, system requiring a novel architecture, or systems with an unorthodox business model or governance model. Copyright © 2011 John Wiley & Sons, Ltd.

Received 28 July 2011; Accepted 29 July 2011

KEY WORDS: agile software development; software development process; process adaptation

1. INTRODUCTION

1.1. Agility defined

Drawing from earlier definitions from Jim Highsmith [1] or from Steve Adolph and the OODA loop [2], we define *agility* as “the ability of an organization to react to changes in its environment faster than the rate of these changes.” This definition uses the ultimate purpose or function of being agile for a business, rather than defining agility by a labeled set of practices (e.g., you’re agile when you do XP [3], Lean [4–6], or Scrum [7–9]) or by a set of properties defined in opposition to another set—the agile manifesto approach [10]. This definition is not too far from that of Kieran Conboy arrived at in his survey of agile process development literature [11].

An analogy could be the definition of a road. Would you define a road as something made of crushed rocks and tar, or define it as a surface that is black rather than white, flat rather than undulated, and with painted lines rather than monochrome? Or would you rather define a road as a component of a transportation system, allowing people and goods to be moved on the ground surface from point A to point B? And then let the properties or components of the road be derived from this functional definition, allowing some novel approaches in road design, rather than defining it narrowly using a common recipe.

Note also that it is quite possible to adopt a labeled set of agile practices: Crystal [12], SCRUM, FDD [13], DSDM [14,15], or a set of practices that perfectly conform to the agile manifesto [10] and not become agile. You would then “do agile”, but you are not agile.

^{*}Correspondence to: Philippe Kruchten, Department of Electrical and Computer Engineering, University of British Columbia, Vancouver, BC, Canada.

[†]E-mail: pbk@ece.ubc.ca

1.2. Agile methods successes

Agile software development methods do undoubtedly succeed in contexts that are identical or very similar to the contexts in which they have been initially created. As these contexts—the “agile sweet spot” [16]—are very frequent in software development, representing more than 70% of all software being developed, this may have sometimes led their proponents to a certain complacency: thinking that their method has universal value, that it represents some ultimate recipe, the holy grail of software engineering.

Agile methods may fail in various ways when they are applied “out of the box”, that is, with no or little adaptation, in contexts that are very far, at least on some dimension, from the context in which they have been originally created. Rather than an analysis of the root cause, this usually triggers screams of “you must have not done it right” by its proponents. And this again leads to discussions of “purity”, “scrumbuts”, and so on [39].

Agile methods can be stretched with variable success outside of the context in which they have been created; for example, scaling them up to larger projects, or across distributed teams. In our experience, the contextual factors that have the greatest risks of derailing agile projects are:

- Size
- Large systems with a lack of architectural focus
- Software development not driven by customer demand
- Lack of support from surrounding stakeholders, traditional governance
- Novice team
- Very high constraint on some quality attribute (safety-critical system, real-time constraints).

As noted by many authors in the last few years [17,18], we cannot just rely on acts of faith by eloquent process gurus to help us define the adequate process, or set of practices outside of the agile sweet spot. Cold-headed, impartial investigation is required. Such research is generally not very easy to conduct; it is often qualitative, rather than quantitative, it draws more from social sciences than computer science, not easy to publish, not easy to carve down to masters’ thesis bite size.

1.3. Overview of this paper

In this paper, drawing from our experience at Rational Software and, subsequently, at KESL in consulting engagements with several organizations that have attempted to transition to agile software development methods, we present a contextual framework or model for situating agile practices. We define several factors characterizing software development context, which affect significantly the adoption of agile methods, especially when projects are outside of the “agile sweet spot” in which these methods have been defined and in which they operate at best. We describe four organizations that are developing software outside of this “sweet spot” and have run into difficulties applying agile software development methods “out of the box”. We compare our model with other similar proposals. The ultimate purpose of this work, however, is to provide means for organizations to rapidly configure their method, by providing guidance on what agile practice to use in which circumstances.

2. CONTEXT DEFINED

Real estate agents in North America will tell you that only three factors do matter in their business: “location, location, and location.” For software process, we claim that only three factors matter: “context, context, and context.” In *Voyage in the Agile Memeplex* [19], we had stressed the necessity to put our processes in context but did not define what “context” meant. It is very unfortunate that too many advocates of agile development practice are preaching good practices but completely removed from the context in which they were proven to be successful. It turns some of their followers, several levels of transmission down to become just blind bigots, sometimes rabid bigots (see [19] for this phenomenon and the down side of *decontextualization*).

2.1. Two levels of context

There are two sets of factors that make up the context, which can be partitioned roughly in two sets: factors that apply at the level of whole organization/company, and factors that apply at the level of the project. In small organizations, with few software development projects, this distinction does not apply, and all factors are on the same level.

The organization-level factors (environment conditions) do influence heavily the project-level factors, which in turn should drive the process and practices that should be used

2.2. Organizational level: environmental factors

A certain number of factors are attached to the organization developing software, more than the particular software development project; see Figure 1.

1. Business domain

For what domain of activity is this organization developing software? Web-based systems, aerospace embedded systems, small hand-held instrumentation? Is software development the primary business activity of the organization, or at the other end, are we dealing with the IT organization of a business for which software is not at all the primary output. This factor, more than any of the other four, will condition, or constrain many of the project-level factors. For example, aerospace or biomedical instrumentation projects tend to be safety-critical.

2. Number of instances

How many instances of the software system (large or small) will be actually deployed? Are you building one single system, a dozen, a thousand, or millions? One-off systems are often internal to an organization, or developed on demand by a system integrator.

3. Maturity of organization

How long has that organization been developing software? How mature are the processes (and the people) relative to software development? Is the organization a small start-up, an SME (small or medium enterprise), or a large multinational firm? A small start-up is more likely to focus on a single, commercial piece of software, or more rarely open-source.

4. Level of innovation

How innovative is the organization? Are you creators or early adopters of new ideas and technologies? Or, treading on very traditional grounds? Large IT organizations, or government agencies tend to be rather risk-adverse and rarely tread on uncharted territory.

5. Culture

In which culture are the projects immersed? Are we speaking here of both national culture and corporate culture? What are the systems of values, beliefs, and behaviors that will impact, support, or interplay with the software development practices [20–24]?

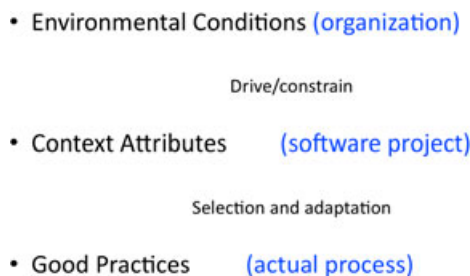


Figure 1. Environmental and project level context attributes.

2.3. Project-level context attributes: the octopus model

At the level of a given software development project, we have identified empirically eight key factors; see Figure 2:

1. Size

The overall size of the system under development is, by far the greatest factor, as it will drive in turn the size of the team, the number of teams, the needs for communication and coordination between teams, the impact of changes, and so on [25,26]. Number of person-months, or size of the code, or development budget are all possible proxies for the size.

2. Stable architecture

Is there an implicit, obvious, *de facto* architecture already in place at the start of the project? Most projects are not novel enough to require a lot of architectural effort. They follow commonly accepted patterns in their respective domain. Many of the key architectural decisions are done in the first few days, by choice of middleware, operating system, programming languages, and so on [27]. Some proponents of agile methods dismiss architecture at Big Up-Front Design, or YAGNI (You Ain't Gonna Need It) [17,28,29], or confine it to a simple explanatory metaphor [3], which is good but not always sufficient.

3. Business model

What is the money flow? Are you developing an internal system, a commercial product, a bespoke system on contract for a customer, a component of a large system involving many different parties? Is it free, libre, and open-source software (FLOSS)?

4. Team distribution

Linked often to the size of the project, how many teams are involved and are not collocated? This increases the need for more explicit communication and coordination of decisions, as well as more stable interfaces between teams and between the software components that they are responsible for [30,31]. Open-source development very often deals with scattered individuals, not teams.

5. Rate of change

Although agile methods are “embracing changes”, not all domains and system experience a very rapid pace of change in their environment. How stable is your business environment

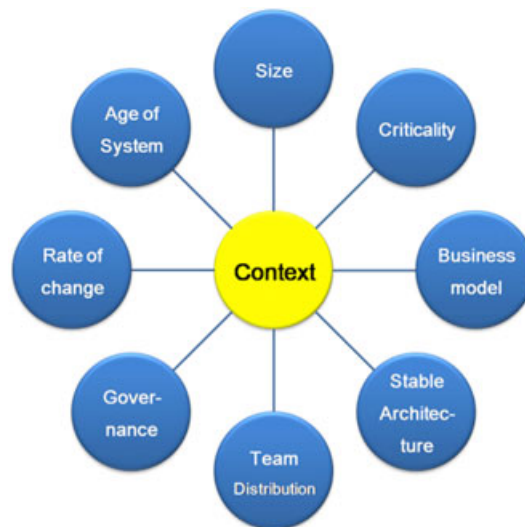


Figure 2. The octopus: eight key contextual factors for Agile software development.

and how much risks (and unknowns) are you facing? There are still projects with very stable requirement definitions.

6. Age of system

Are we looking at the evolution of a large legacy system, bringing in turn many hidden assumptions regarding the architecture, or the creation of a new system with fewer constraints?

7. Criticality

How many people die or are hurt if the system fails? Documentation needs to increase dramatically to satisfy external agencies who will want to make sure that the safety of the public is assured [32–35].

8. Governance

How are projects started, terminated? Who decides what happens when things go wrong? How is success or failure defined? Who manage the software project managers? [36–38]

2.4. Relationship between the two sets of factors

As you can expect, the first set of factors (organizational level) impacts and constrains the second set as shown on Figure 3. But there is still a wide range of variation in this second set inside any given organization, especially large software development shops (a.k.a. system integrators) offering “bespoke” software development services.

If your business domain is “aerospace-onboard software”, this will precondition several project-context factors: criticality, size, business model, and so on. This in turn will make some practices suitable or not, will influence amount and type of documentation, the “level of ceremony” required by a given project.

2.5. The agile sweet spot

Figure 4 shows, based on this model, the “agile sweet spot,” that is, the conditions under which most “labeled” agile software development have been developed, and for which success is pretty much

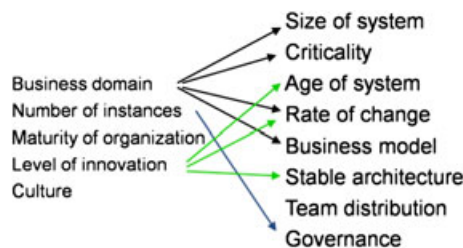


Figure 3. Relationships between the two sets.

System Size • 0 ..12 ... 300
 Criticality • Simple, \$ losses, ... deaths
 System Age • Exploratory, greenfield, legacy maintenance
 Rate of change • Low, medium, high
 Business model • In house, Open source,
 Stable architecture • Stable, changed, new
 Team distribution • Collocated, ..., offshore outsource
 Governance • Simple rules, ..., SOX, ...

Figure 4. A particular context: the Agile sweet spot (in blue).

assured [16]. This would be the case, for example, of a web-based e-commerce site, built on dot-net technology by a small team, interacting with their customers.

The “agile sweet spot” tends to be for collocated team, of less than 15 people, doing greenfield development for non-safety-critical system, in rather volatile environment; the system architecture is defined and stable, and the governance rules straightforward. Again, we are not trying to say that agile practices do not work outside of this sweet spot, but that many are challenged, will need some adaptation, and, in some cases, not be suitable.

3. FOUR AGILE PROJECTS OUTSIDE OF THE SWEET SPOT

In our consulting practice, we have come across four organizations in the last 10 years that have tried to embrace agile software development methods but have run into difficulties [39]. We found, in many cases, that this was mostly because they were, in some way, outside of the “agile sweet spot”.

1. Project FIN

This large legacy software was being re-implemented in Java: 50 developers collocated for about 18 months. Although progressing very well for the first 6 months using a combination of XP and Scrum, this project “hit a wall” after a while, mostly because of its failure to put in place a solid underlying architecture, driven by a naïve belief that a solid software architecture would gradually emerge as the result of bi-weekly refactorings.

2. Project FAC

A large factory automation project, with again a large amount of legacy software, felt outside of the sweet spot on several other dimensions: safety-critical, very low rate of change, and no concept of “customer” to interact with, because many aspects are just related to physics. Software cannot be tested in real environment, and there is only one release opportunity per year.

3. Project FLY

Multiple interrelated legacy and “greenfield” projects, some deployed in the cockpit of aircraft, having to comply to the highest safety critical standards [32], and therefore requiring large amount of very detailed documentation, in particular traceability of artifacts, which run counter to the agile manifesto [10], created cultural clashes in the development teams.

4. Project ANA

Small start-up company developing novel algorithms to support security trading, not driven by customers’ demand but driving the development internally based on models from physics, hence put many of the agile practices at odds.

In each of these four projects, the way forward was defined by the following pattern:

1. Develop a clearer understanding of the contextual factors, using our model, for example, allowed the project(s) to focus on what was specific to their project,
2. and specify the areas needing improvement from a software process perspective,
3. and finally, select carefully the agile practices that would solve the issues they actually were facing, rather than blind application of a labeled agile methods in its totality, all practices lumped together, for example, all XP practices.

4. REVIEW OF OTHER SIMILAR CONTEXTUAL MODELS

Other authors have attempted to define the *context* of a software process, to define the main factors that affect the process they use/could use/would use. Let us examine a few such models in the agile adoption arena:

1. Boehm–Turner

In their book, *Balancing Agility and Discipline* [40], Barry Boehm and Richard Turner used five factors to contrast software process/methods between what they call “plan-driven methods” and agile methods:

1. Size,
2. Criticality,
3. Personnel (their skill, know-how),
4. Dynamism (rate of change), and
5. Culture (of the team: thriving on chaos or on order). Their five factors provided us with starting point, but we found that: (i) they were mixing organization-level issues with project-level issues; and (ii) they were lumping together too many aspects into the factors Personnel and Culture.

2. Cockburn and Crystal

In the Crystal family of processes [12], Alistair Cockburn defines different processes based on the following:

1. Size,
2. Criticality, and
3. Skills. The first two are fully aligned with ours, but we found the “Skills”, which matches “Personnel” in Boehm–Turner, difficult to use in practice. It is not quite a linear scale. We do capture some of it under “Maturity of the organization” however.

3. Ambler (IBM) and “Agile@Scale”

Closer to our views are those of Scott Ambler in *Agility at Scale* [41], which is summarized by his table reproduced in Figure 5. We seem to cover mostly the main grounds, although with some small differences here and there, as shown in Table I. Scott Ambler seems to focus on scaling agility to larger projects, whereas I am mostly attempting to adjust to the context, regardless of the size or ambition.

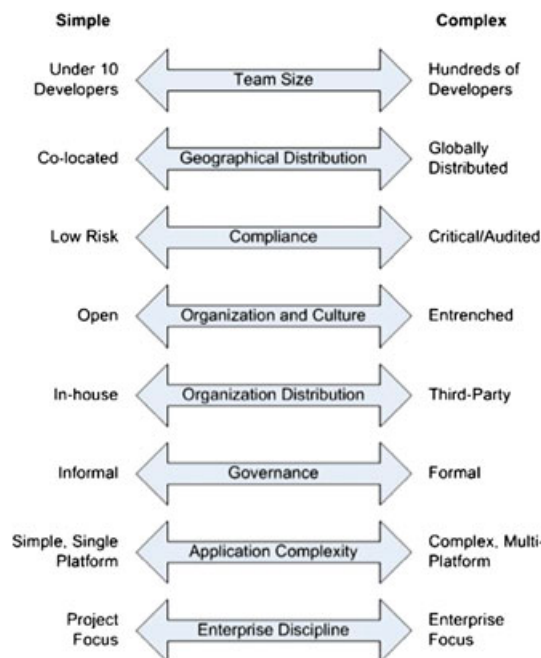


Figure 5. Scott Ambler's scaling factors (from Figure 4 in [41]).

Table I. Comparing Agile and Scale [41] and our contextual model.

<i>Ambler</i>	<i>Kruchten</i>	<i>Comment</i>
Team size	Size	Number of people or SLOCs or function points? Either one is a possible indicator of size, as they are strongly correlated.
Geographical distribution	Team distribution	(Same concept)
Compliance	Criticality	Not quite identical, but linked: critical system have to be compliant to standards and regulations: SOX [42], Cobit, [43], Basel II [44], or Do178B [32]
Organization and culture	<i>Culture</i>	depending on the industry
Organization distribution	Business model	Rarely specific to one project
Application complexity	Stable architecture, size	(Same concept)
Enterprise discipline	<i>Maturity of the organization</i>	Could be also linked to innovation level
Governance	Governance	May vary across projects
	Age of system	(Same concept)
	Rate of change	Issues with large legacy system
		Drives iteration duration, and the nature of the feedback loop

Table II. Guidance for process configuration.

<i>Factor</i>	Size			Criticality			Distribution				Rate of change		Age of system		...
<i>Values</i>	S	M	L	L	M	H	-	M	L	XL	L	H	G	B	
Iterations															
Daily standup															
Retrospective															
Pair prog.															
Backlog															
Metaphor															
Monthly release to users															
More agile practices...															

Legend:

	Practice is not affected by the factor
	Practice needs caution or adaptation
	Practice not useful, or require considerable adaptation
	Practice to avoid, could be dangerous or counterproductive
	Not enough evidence to conclude

5. FUTURE WORK

The authors and several colleagues have used the model in an *ad hoc* fashion, mostly applying judgement calls and past (negative) experience to decide which practice could be used in a given combination of project factors. The ideal situation would be to build a kind of recommending system: a tool to which we would provide values for the 5+8 factors that would give an indication of which practices are usable, which are not, which would require adaptation or special consideration, as the starting point for a process configuration. The main hurdle is to objectively populate such a table with reliable data, supported by evidence, and not just guts' feelings, as the number of data points is rather large. A short list of agile practices would be about 40: short iterations, managing a backlog, daily stand-up meetings, test-driven development, continuous integration, and so on. With only three or four values for each of the eight factors in the "octopus", we have already a thousand data points. Some cases are easier than others: short iterations, or conducting retrospectives have rather wide applicability. Table II shows an embryo of what such a table could contain.

Even in this simple form, the table generates lots of discussions and controversies, and each non-green cell must have an explanation, or rationale, and some caveats. So, it should not be used as some kind of prescription, but more as a way to focus the reflection and discussion on the value of practices. Fortunately, as agile methods have crossed the chasm and become mainstream, there is a growing body of evidence to draw from, in particular experience reports, such as the ones collected through agile conference [45,46], even though many tend to be heavily biased towards the "green", and complemented by studies of researchers, looking at the impact of going outside the sweet spot [47], or just plain old critiques of agile methods [48]. It should be useful to also submit non-agile practices to the same filter.

6. CONCLUSION

In assisting software organization adopting agile software development processes, we found valuable to first define the context using our model with several factors, then understand in which dimension(s) the project felt outside of an ideal agile sweet spot, and then in turn drive the adoption and possible adaptation of agile practices to this context, rather than a forced-fit, en bloc adoption of all practices falling under a certain agile label, in the sometimes naïve hope that it will cure all ills. But again, agility should not be defined in terms of practices, but as the ability of an organization to react to changes in its environment faster than the rate of these changes.

Since its presentation at EuroSPI 2010 [49], this context model has been integrated in a more encompassing conceptual model of software development project called "the frog and the octopus" [50].

REFERENCES

1. Highsmith JA. Agile Software Development Ecosystems. Addison-Wesley: Boston, 2002.
2. Adolph WS. What lessons can the agile community learn from a maverick fighter pilot? Melnik, G (ed.). Agile 2006 Conference, Minneapolis, MN, USA. IEEE Computer Society, Los Alamitos 2006; 94–99.
3. Beck K. Extreme Programming Explained. *Embrace Change*. Addison-Wesley: Boston, 2000.
4. Poppendieck M, Poppendieck T. Lean Software Development – An Agile Toolkit. Addison-Wesley: Boston, 2003.
5. Poppendieck M, Poppendieck T. Implementing Lean Software Development – From Concept to Cash. Addison Wesley: Upper Saddle River, NJ, 2009.
6. Poppendieck M, Poppendieck T. Leading Lean Software Development. Results Are not the Point. Addison-Wesley Professional: Boston, 2009.
7. Cohn M. Succeeding with Agile-Software Development Using Scrum. Pearson Education: Boston, 2009.
8. Schwaber K, Beedle M. Agile Software Development with SCRUM. Prentice-Hall: Upper Saddle River, NJ, 2002.
9. Schwaber K. The Enterprise and Scrum. Microsoft Press: Redmond, WA, 2007.
10. Agile Alliance. Manifesto for Agile Software Development, 2001. <http://www.agilemanifesto.org/> [9 May 2011]
11. Conboy K. Agility from first principles. Reconstructing the concept of agility in information systems development. *Information Systems Research* 2009 **20**: 329–354. <http://hdl.handle.net/10344/574> [9 May 2011]
12. Cockburn A. Crystal Clear – A Human-Powered Methodology for Small Teams. Addison-Wesley: Boston, 2004.

13. Palmer SR, Felsing JM. A Practical Guide to Feature-Driven Development. Prentice-Hall: Upper Saddle River, NJ, 2002.
14. Stapleton J. DSDM, Dynamic Systems Development Method. The Method in Practice. Addison-Wesley: Reading, 1998.
15. Stapleton J (ed.). DSDM Business Focused Development. Addison-Wesley: London, UK, 2003.
16. Kruchten P. Scaling down projects to meet the Agile sweet spot. *The Rational Edge* August 2004. <http://www-106.ibm.com/developerworks/rational/library/content/RationalEdge/aug04/5558.html> [9 May 2011]
17. Abrahamsson P, Ali Babar M, Kruchten P. Agility and architecture. Can they coexist? *IEEE Software* 2010 **27** (2):16–22. DOI: 10.1109/MS.2010.36
18. Abrahamsson P, Salo O, Ronkainen J, Warsta J. Agile Software Development Methods-Review and Analysis. VTT Technical Research Centre of Finland: Oulu, Finland 2002.
19. Kruchten P. Voyage in the Agile Memeplex. Agility, Agilese, Agilitis, Agilology. *ACM Queue* 2007 **5**(5): 38–44. DOI: 10.1145/1281881.1281893. <http://queue.acm.org/detail.cfm?id=1281893> [9 May 2011].
20. Borchers G. The Software Engineering Impacts of Cultural Factors on Multi-cultural Software Development Teams. *25th International Conference on Software Engineering (ICSE'03)* Portland, OR. IEEE Computer Society, 2003; 540–545
21. Sharp H, Woodman M, Hovenden F, Robinson H. The Role of 'Culture' in Successful Software Process Improvement. *25th Euromicro Conference (EUROMICRO '99)*, Vol. 2. IEEE-CS, Milan, Italy, 1999; 2170–2176.
22. Hsieh Y, MacGregor EL, Kruchten P. Intercultural factors in global software development. In: Kasap S (ed.). 18th Annual Canadian Conference on Electrical and Computer Engineering (CCECE'05). Saskatoon, SK, IEEE Canada 2007.
23. Kruchten P. Analyzing intercultural factors affecting global software development. In: Damian D, Lanubile F (eds.). 3rd International Workshop on Global Software Development (GSD2004), Collocated with ICSE 2004, Edinburgh, Scotland, IEEE 2004; 59–62.
24. Sharp H, Robinson H, Woodman M. Software engineering. Community and culture. *IEEE Software* 2000; **17**(1): 40–47. DOI: 10.1109/52.819967.
25. Larman C, Vodde B. Scaling Lean & Agile Development – Thinking and Organizational Tools for Large-Scale Scrum. Addison-Wesley Professional: Boston, 2008.
26. Leffingwell D. Scaling Software Agility. Best Practices for Large Enterprises. Addison-Wesley Professional: Boston, 2007.
27. Kruchten P. The Rational Unified Process. An Introduction 3rd ed. Addison-Wesley Professional: Boston 2003.
28. Madison J. Agile-architecture interactions. *IEEE Software* 2010 **27**(2): 41–47. DOI: 10.1109/MS.2010.35
29. Blair S, Watt R, Cull T. Responsibility-driven architecture. *IEEE Software* 2010 **27**(2): 26–32. DOI: 10.1109/MS.2010.52.
30. Ågerfalk PJ, Fitzgerald B. Flexible and distributed software processes. Old Petunias in new bowls? *Communications of the ACM* 2006 **49**(10): 27–34.
31. Lings B, Lundell B, Ågerfalk PJ, Fitzgerald B. Ten strategies for successful distributed development. *Proceedings of the IFIP WG 8.6 Conference*, Galway: Ireland. Vol. 206. Springer-Verlag: Berlin, 2006; 94–112
32. RTCA/DO-178B Software Considerations in Airborne Systems and Equipment Certification. Radio Technical Commission for Aeronautics: Washington, DC 1992.
33. Leveson NG. Safeware. System Safety and Computers. Addison-Wesley: Reading, MA, 1995.
34. Paige RF, Chivers H, McDermid JA, Stephenson ZR. High-Integrity Extreme Programming. *Proc. of 2005 ACM Symposium on Applied Computing*. Santa Fe, New Mexico, USA, ACM 2005; 1518–1523.
35. Poppendieck M, Morsicato R. Using XP for safety-critical software. *Cutter IT Journal* 2002 **15**(9): 12–16.
36. Cockburn A. A governance model for incremental, concurrent, or Agile projects. *CrossTalk. The Journal of Defense Software* 2006 **19**(2): 13–17.
37. Dubinski Y, Yaeli A, Feldman Y, Zarpas E, Nechushtai G. Governance of software development. The transition to Agile scenario. In: Cater-Steel A (ed.), IT Governance and Service Management Frameworks and Adaptations, Idea Group Publishing, Information Science Publishing, IIR Press: Hershey, PA 2009; 266–284.
38. Dubinsky Y, Kruchten P. Software development governance (SDG). Report on 2nd ICSE Workshop. *ACM SIGSOFT Software Engineering Notes* 2009 **24**(5): 46–47. DOI: 10.1145/1598732.1598760.
39. Hoda R, Kruchten P, Noble J, Marshall S. Agility in Context, in: Rinard M (ed.). Proceedings of OOPSLA 2010 (part of SPLASH), Las Vegas, 17–21 October 2010, ACM 2010:74–88. DOI: 10.1145/1869459.1869467.
40. Boehm BW, Turner R. Balancing Agility and Discipline – A Guide for the Perplexed. Addison-Wesley: Boston, MA 2003.
41. Ambler SW. Agility at Scale. Become as agile as you can be. IBM: Toronto 2009. <http://www.internetevolution.com/ebook/ebookibm7/> [9 May 2011]
42. US Public Law 107 – 204 - Sarbanes-Oxley Act of 2002. US Government Printing Office: Washington 2002.
43. CobiT 4.0--Control Objectives for Information and related Technology – Control Objectives, Management Guidelines, Maturity Models. IT Governance Institute: Rolling Meadows, IL, USA 2005.
44. Basel II-Revised international capital framework, Bank for International Settlements: Basel, CH 2004.
45. Melnyk G, Poppendieck M (eds.). *Proceedings of Agile 2008 Conference*, Toronto, 4–8 Aug. IEEE Computer Society: Los Alamitos, 2008.
46. Dubinsky Y, Dybå T, Adolph SW, Sidiky A (eds.). Proceedings of Agile 2009 Conference, Chicago, Aug 24–28, IEEE Computer Society: Los Alamitos, 2009.

47. Šmite D, Moe NB, Ågerfalk PJ (eds.). *Agility Across Time and Space – Implementing Agile Methods in Global Software Projects*, Springer-Verlag: Berlin, 2010.
48. Stephens M, Rosenberg D. *Extreme Programming Refactored – The Case Against XP*. Apress LP: Berkeley, CA 2003.
49. Kruchten P. Contextualizing agile software development. Presented at *EuroSPI 2010*, Grenoble, 3 Sept. 2010.
50. Kruchten P. The frog and the octopus: a model of software development. *CSI Communications* 2011 **35**(4): 12–15.

AUTHOR'S BIOGRAPHY



Philippe Kruchten is professor of software engineering at the University of British Columbia in Vancouver, where he holds an NSERC chair in design engineering. He had joined UBC in 2004 after a 32-year career in the software industry, developing systems in telecommunications, defense, and aerospace. His main interests are in software architecture, software project management, and software development processes. During his 17 years with Rational Software (now part of IBM), he led the development of the Rational Unified Process® (RUP®), an iterative and incremental software development process. He is the co-founder and secretary of the IFIP working group 2.10 on Software Architecture, and the co-founder and past chair of Agile Vancouver, a special interest group focusing on agile software development approaches.

Dr Kruchten has a mechanical engineering degree from Ecole Centrale de Lyon (France), and a doctorate degree from Ecole Nationale Supérieure des Télécommunications in Paris. He is a licensed professional software engineer in British Columbia, Canada, an IEEE Certified Software Development Professional (CSDP) and even a Certified Scrum Practitioner (CSP). He is a member of IEEE, ACM, INCOSE, and AIS.